Backstage Local Provisioning System

Design Document

Stratpoint Global Outsourcing, Inc.

1. Executive Summary

This document outlines the architecture and implementation strategy for the Backstage Internal Developer Platform (IDP) at Stratpoint Global Outsourcing, Inc. This platform represents a strategic investment that begins with developer training, expands to internal project delivery, and ultimately positions Stratpoint as a platform services provider. Hosted at

https://portal.stratpoint.io/

the platform enables local resource provisioning with a clear growth trajectory: from training environments to internal best practices enforcement, client replication, and ultimately a service offering for external organizations.

1.1 Objectives

- Enable self-service provisioning of local development resources through cloud-hosted Backstage
- Provide standardized training environments for developers
- Maintain centralized governance and tracking of all provisioned resources
- Support initial use case: Kafka instance provisioning for training
- Establish golden paths and best practices for Stratpoint internal projects
- Create replicable platform model for client organizations
- Position platform as potential service offering for external organizations

1.2 Scope

This platform follows a phased strategic approach: Phase 1 focuses on training environments (Kafka provisioning for developer learning), Phase 2 establishes internal golden paths and best practices templates, Phase 3 replicates the model for client organizations, and Phase 4 positions it as a service offering for external markets. The architecture is designed to scale across all these phases with minimal modification.

1.3 Strategic Roadmap

The platform development follows a strategic four-phase progression, each building on the previous phase while adding significant business value:

Phase 1: Learning and Training Foundation

- **Timeline:** 10 weeks (initial implementation)
- **Focus:** Developer training environments, starting with Kafka provisioning
- **Business Value:** Reduced training setup time from hours to minutes, standardized learning environments, improved training quality and consistency
- **Deliverables:** Core platform architecture, agent system, Kafka template, catalog integration

Phase 2: Internal Golden Paths

- **Timeline:** 3-6 months after Phase 1
- **Focus:** Stratpoint best practices, standardized project templates, composite stacks, AI-native development integration
- **Business Value:** Faster project delivery, consistent quality across teams, reduced onboarding time for new projects, enforcement of security and compliance standards
- **Deliverables:** Full-stack templates, microservice boilerplates, data pipeline environments, AI development configurations

Phase 3: Client Platform Replication

- **Timeline:** 6-12 months after Phase 2
- **Focus:** Deploy customized Backstage IDP instances for client organizations, multi-tenancy support
- **Business Value:** Competitive differentiator, increased client satisfaction, higher project margins, strategic client relationships
- **Deliverables:** Client deployment automation, white-label capabilities, client-specific template libraries

Phase 4: Platform-as-a-Service Offering

- **Timeline:** 12-18 months after Phase 3
- **Focus:** Commercial platform offering for external organizations, SaaS deployment model, marketplace of templates
- **Business Value:** New revenue stream, market positioning as platform vendor, recurring revenue model, scalable business growth
- **Deliverables:** Multi-tenant SaaS architecture, billing integration, marketplace platform, customer success infrastructure

Note: Each phase builds on the previous foundation. The core architecture designed in Phase 1 is intentionally scalable to support all subsequent phases without major refactoring.

2. System Architecture

2.1 High-Level Architecture

The system follows a pull-based agent architecture to overcome network connectivity challenges between cloud-hosted Backstage and local developer machines.

Cloud - Backstage Instance https://portal.stratpoint.io/ Scaffolder | Catalog | Task Queue | Plugin API
⇕ HTTPS REST API (Google OAuth)
Local Developer Machine Backstage Agent (npm package) Receives tasks via SSE | Executes | Reports status
⇕ Docker API
Provisioned Resources Kafka | PostgreSQL | Microservices | Containers

2.2 Component Overview

Component	Description
Backstage Scaffolder	Provides self-service templates for resource provisioning
Custom Plugin	Manages task queue, agent API, and catalog integration
Task Queue	Stores pending/in-progress/completed provisioning tasks
Local Agent	npm package running on developer machines, receives tasks via SSE
Software Catalog	Tracks all provisioned resources with metadata and lifecycle

3. System Components

3.1 Backstage Instance

Instance URL: https://portal.stratpoint.io/

3.1.1 Scaffolder Templates

Define self-service workflows for resource provisioning. Example Kafka template structure:

```
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: kafka-local-provision
  title: Provision Kafka Instance Locally
spec:
  parameters:
    - kafkaVersion: 3.6.0
    - ports: 9092
  steps: [queue-task, register-catalog]
```

3.1.2 Custom Backstage Plugin

Plugin Name: @stratpoint/plugin-local-provisioner

Responsibilities:

- Task Queue Management: Stores provisioning tasks in PostgreSQL/Redis
- Agent API: Provides REST endpoints for agent SSE connection and status updates
- Catalog Integration: Auto-registers provisioned resources in Software Catalog
- Lifecycle Management: Tracks resource states and health

3.1.3 Software Catalog

Tracks all provisioned resources with metadata including owner, configuration, status, and dependencies.

3.2 Local Agent

Package: @stratpoint/backstage-agent

Distribution: npm global package

3.2.1 Core Functions

- Authentication: Google OAuth token from Backstage
- Task Reception: Receives tasks via Server-Sent Events (SSE) in real-time

- Execution Engine: Runs Docker Compose, scripts, or containerized workflows
- Status Reporting: Updates Backstage with progress and completion status
- Health Monitoring: Reports resource health metrics

3.2.2 Installation & Setup

```
# Install agent globally
npm install -g @stratpoint/backstage-agent

# Authenticate with Backstage
backstage-agent login --url https://backstage-stratpoint.rcdc.me --google

# Start agent (runs as background service)
backstage-agent start
```

3.3 Task Queue System

Storage: PostgreSQL or Redis

Schema:

Field	Type	Description
task_id	UUID	Unique task identifier
agent_id	UUID	Target agent for execution
task_type	String	provision-kafka, provision-postgres, etc.
config	JSON	Resource configuration parameters
status	Enum	pending, in-progress, completed, failed
created_at	Timestamp	Task creation time
updated_at	Timestamp	Last status update

4. Technical Requirements

4.1 Technology Stack

Component	Technology	Version
Backstage	Spotify Backstage	Latest LTS
Backend API	Node.js + Express	Node 18+
Database	PostgreSQL	14+
Cache (Optional)	Redis	7+
Local Agent	Node.js + TypeScript	Node 18+
Containerization	Docker + Docker Compose	Docker 20+, Compose v2
Authentication	Google OAuth 2.0	Latest

4.2 Infrastructure Requirements

4.2.1 Cloud Infrastructure

- Backstage Instance: Existing at https://portal.stratpoint.io/
- Database: PostgreSQL for task queue and catalog
- Optional: Redis for high-performance task queuing
- API Endpoints: RESTful APIs for agent communication

4.2.2 Local Developer Machine

- Node.js 18+ (for agent runtime)
- Docker Engine 20+ (for containerized resources)
- Docker Compose v2+
- Minimum 4GB RAM, 10GB free disk space
- Internet connectivity for agent SSE connection

4.3 Security Requirements

- Google OAuth-based authentication for agent-to-Backstage communication

- TLS 1.3 for all API communications
- Role-based access control (RBAC) for template access
- Audit logging of all provisioning activities
- Token rotation every 90 days

5. Implementation Plan

Phase	Duration	Deliverables	Key Activities
Phase 1	2 weeks	Plugin scaffolding, API design, database schema	Setup dev environment, design APIs
Phase 2	3 weeks	Local agent MVP, authentication, SSE task reception	Build agent, implement auth
Phase 3	2 weeks	Kafka template, Docker Compose configs	Create Kafka scaffolder template
Phase 4	2 weeks	End-to-end testing, documentation	Integration testing, user docs
Phase 5	1 week	Pilot with 10 developers	Pilot rollout, gather feedback
Phase 6	Ongoing	General availability, additional templates	Production rollout, expand resources

6. User Workflows

6.1 Initial Setup Workflow

1. Developer installs agent: npm install -g @stratpoint/backstage-agent
2. Developer authenticates with Backstage using Google SSO
3. Agent registers with Backstage and receives unique agent ID
4. Agent establishes SSE connection for real-time task delivery

6.2 Kafka Provisioning Workflow

5. Developer navigates to https://portal.stratpoint.io/create
6. Selects Provision Kafka Instance Locally template
7. Fills in parameters: Kafka version, ports, retention policies, topic configurations
8. Clicks Create - Backstage queues provisioning task
9. Backstage pushes task to agent via SSE
10. Agent downloads Docker Compose configuration
11. Agent executes: docker-compose up -d
12. Agent validates Kafka is running and healthy
13. Agent reports success to Backstage
14. Backstage creates catalog entry for the Kafka instance
15. Developer views Kafka instance in Software Catalog with connection details

6.3 Lifecycle Management

- **Start:** Agent starts containers automatically
- **Stop:** Developer can stop via Backstage UI or local Docker commands
- **Health Monitoring:** Agent reports health status every 5 minutes
- **Decommission:** Backstage UI triggers cleanup task, agent removes containers

7. API Specifications

7.1 Agent Authentication (Google OAuth)

```
POST /api/agent/auth
Request: { "googleToken": "google-oauth-token" }
Response: { "serviceToken": "backstage-service-token", "agentId": "agent-uuid" }
```

7.2 Task Reception (Server-Sent Events)

```
GET /api/agent/events?agentId=<id>
Content-Type: text/event-stream
event: task
data: {"taskId":"uuid","type":"provision-kafka","config":{"version":"3.6.0","ports":9092}}
```

7.3 Status Update

```
PUT /api/agent/tasks/:taskId/status
Request: {
  "status": "completed",
  "metadata": { "host": "localhost", "port": 9092 }
}
```

8. Monitoring & Observability

8.1 Metrics

- Task completion rate and duration
- Agent connectivity and uptime
- Resource health status
- API latency and error rates

8.2 Logging

- Centralized logging in Backstage for all provisioning activities
- Local agent logs stored in ~/.backstage-agent/logs
- Audit trail for compliance and debugging

9. Strategic Growth Path

The following capabilities support the strategic phases outlined in Section 1.3. While Phase 1 focuses on training (Kafka provisioning), the architecture is designed to seamlessly accommodate Phases 2-4 through the extensibility patterns described below. Each capability directly enables specific business outcomes across the growth trajectory.

9.1 Planned Resources

- PostgreSQL databases
- Redis instances
- MongoDB clusters
- Microservice templates (Node.js, Python, Java)
- Local Kubernetes clusters (k3s, minikube)
- Development tools (Elasticsearch, RabbitMQ)

9.2 Composite Stack Templates

One of the most powerful future capabilities is composite templates that provision entire application stacks with all dependencies in a single operation. Instead of provisioning individual resources separately, developers can deploy complete, pre-configured environments.

- **E-commerce Microservice Stack:** Kafka + PostgreSQL + Redis + Node.js API + Nginx, all connected and configured
- **Data Pipeline Environment:** Kafka + Apache Spark + PostgreSQL + Airflow, fully orchestrated
- **Full-stack Training Application:** Database + Backend API + Frontend + Message Queue with sample data
- **Event-driven System:** Kafka + Schema Registry + ksqlDB + Kafka Connect + monitoring tools

Key Capabilities:

- Multi-resource orchestration with dependency management
- Automatic service startup ordering (databases before applications)
- Pre-configured integrations with auto-populated connection strings
- Network configuration and service discovery setup
- Sample data and configuration for immediate productivity
- One-click deployment and teardown of entire environments

This transforms the platform from a resource provisioning tool into a complete development environment platform, dramatically reducing setup time from hours to minutes and ensuring consistency across all developer machines.

9.3 AI-Native Development Integration

As software development shifts towards AI-driven SDLC and AI-native development practices, the platform can evolve to integrate AI coding assistants and agentic development tools directly into provisioned environments. This extends the platform beyond training scenarios into production development workflows.

- **Claude Code Integration:** Pre-configure provisioned environments with Claude Code for agentic coding tasks, automated code generation, and AI-assisted development
- **Cursor and AI Assistants:** Templates can include workspace configurations for Cursor, GitHub Copilot, and other AI coding tools with project-specific context
- **AI Agent Workflows:** Provision environments with pre-configured AI agents for testing, code review, documentation generation, and deployment automation
- **Production-Ready Templates:** Move beyond training to create production development environments with AI-powered boilerplates, scaffolding, and best practices built-in

Capabilities:

- AI assistant configurations embedded in project templates and boilerplates
- Context files, system prompts, and AI agent instructions pre-configured for specific tech stacks
- Integration with Claude Code CLI for terminal-based agentic development
- Automated setup of AI-powered development workflows (code generation, review, testing)

- Standardized AI development practices across teams and projects
- Version-controlled AI configurations as part of the project scaffold

This positions the platform at the forefront of AI-native development, enabling teams to leverage cutting-edge AI coding tools from day one. Templates become intelligent starting points that combine infrastructure, application code, and AI development workflows in a single provisioning operation.

9.4 Integration Opportunities

- CI/CD pipelines for local testing
- Cost tracking and resource utilization
- Integration with cloud environments for hybrid workflows
- Team collaboration features

10. Appendices

Appendix A: Sample Kafka Docker Compose

```
version: '3.8'
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - '9092:9092'
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
```

Appendix B: Agent Configuration

```
# ~/.backstage-agent/config.yml
backstage:
  url: https://backstage-stratpoint.rcdc.me
  token: <google-oauth-token>

agent:
  id: <agent-uuid>
  reconnect_timeout: 5  # seconds

logging:
  level: info
  path: ~/.backstage-agent/logs
```

Appendix C: Required Tools & Dependencies

Tool	Purpose	Installation
Node.js	Runtime for agent and Backstage	https://nodejs.org
Docker	Container runtime for Kafka and resources	https://docker.com
PostgreSQL	Database for Backstage and task queue	https://postgresql.org
TypeScript	Development language for agent	npm install -g typescript
Backstage CLI	Plugin development and testing	npm install -g @backstage/cli

Tool	Purpose	Installation
Yarn	Package manager for Backstage	npm install -g yarn
Git	Version control	https://git-scm.com
Redis (Optional)	High-performance task queue	https://redis.io

Document Version: 1.0

Last Updated: December 2025
Prepared by: Stratpoint System Engineering Team